Memory management and isolation

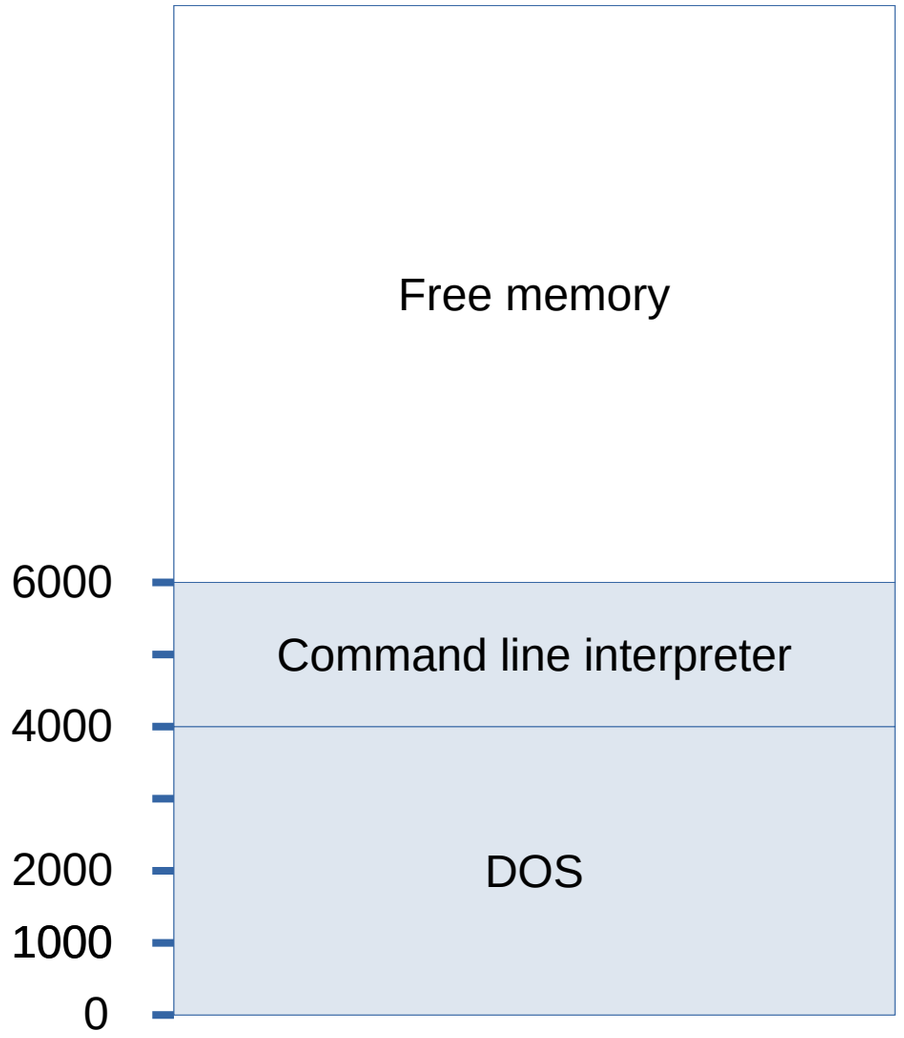
- Isolation is a key ability of virtualization
- It is not simple to implement and require cooperation between
  - The operating system (or hypervisor), that is aware of and manage the processes
  - The hardware (the CPU, and especially its Memory Management Unit – the MMU) that effectively accesses the memory.

Introduction to memory management

- Memory management is a complex topic, and starting with a (very) simplified model is recommended.
- The memory can be seen as a very big one-dimensional bytes array.
  - The first byte of memory is located at address 0, the second byte at address 1, etc.
  - This goes on up to addressing the totality of the memory (a few billions bytes in most modern computers).

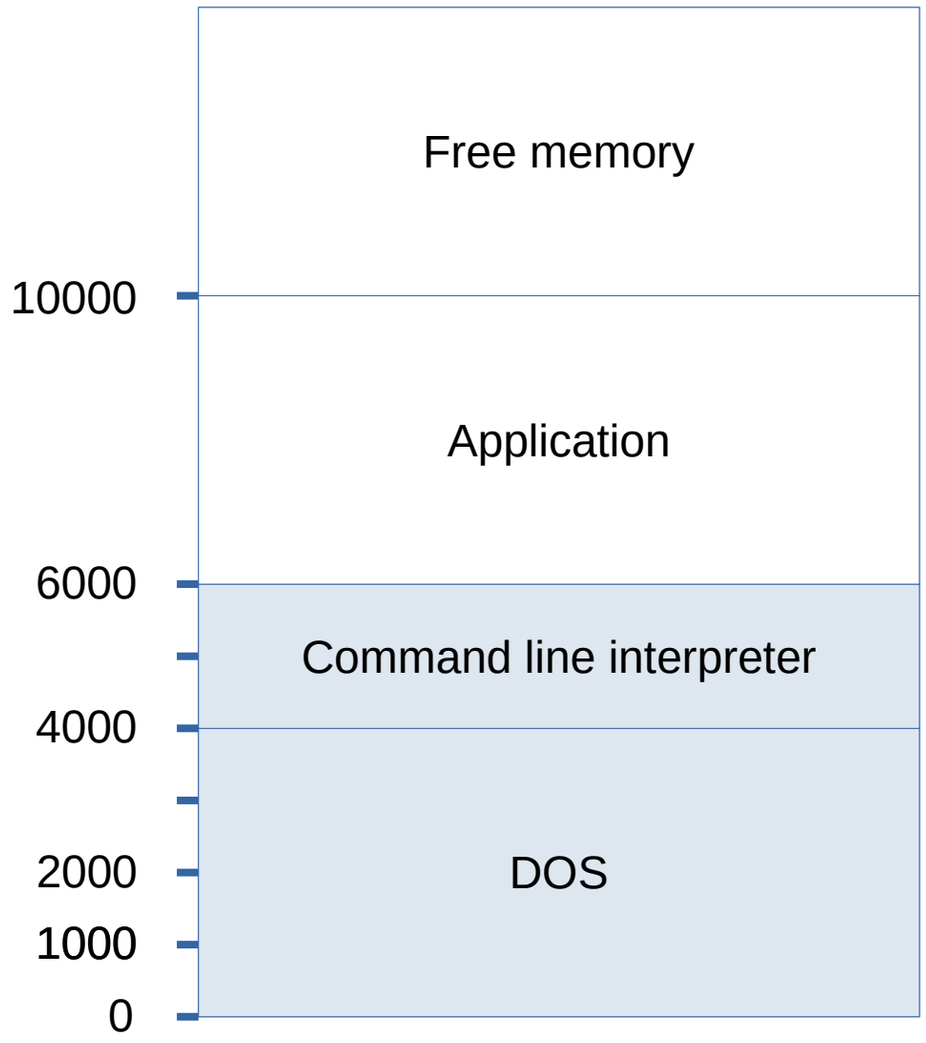
State after booting

- When booting, the Operating System's kernel is loaded into memory from the disk.
- In MS-DOS case, it occupies the lowest addresses.
- Then the command line interpreter is loaded into memory, in the immediately available free space.
- Any remaining memory (at upper addresses) stays free to use.



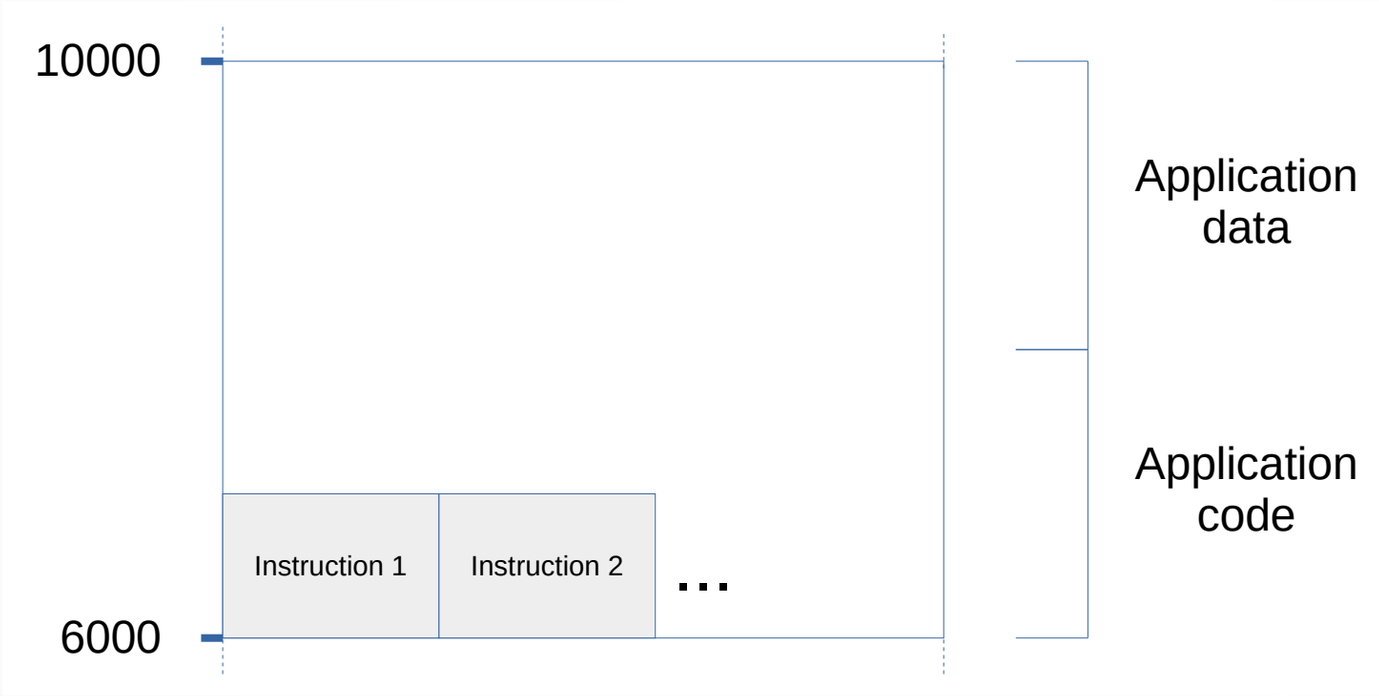
State after booting

- The user can start a new application from the command line.
- The application code is read from its executable file and loaded into memory.
- Some free space is still available at the upper addresses.



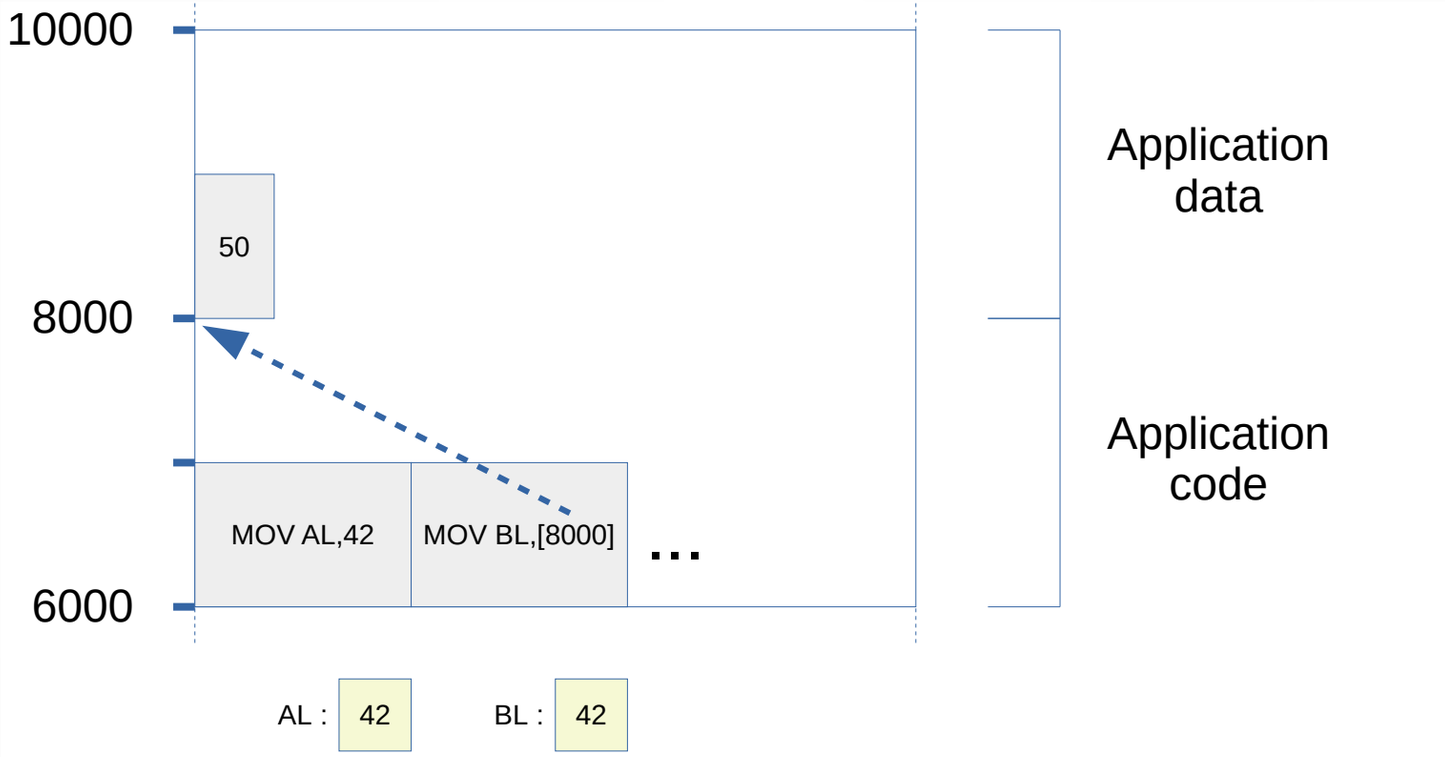
Detail of an application

- Space allocated to an application is divided into two parts:
  - Code, where the list of instructions resides. Each instruction occupies a few bytes, and they come in sequence.
  - Data, where the variables and other allocated data resides.



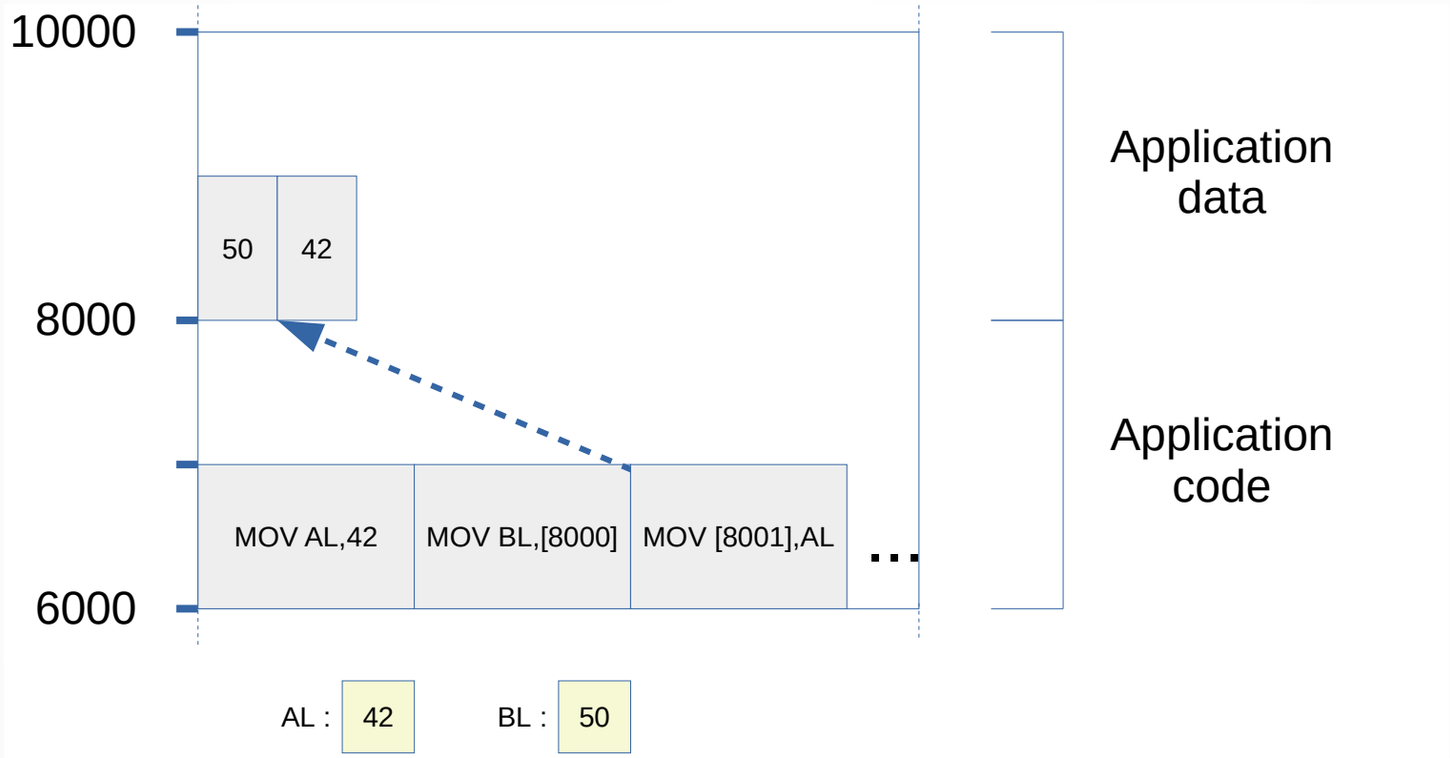
Reading from memory

- Each instruction is executed in order, starting at the beginning of the code area.
  - MOV AL,42 put the value 42 into the AL register
  - MOV BL,[8000] look at address 8000 (inside the data area), take the value (here:50) and put it into BL register.



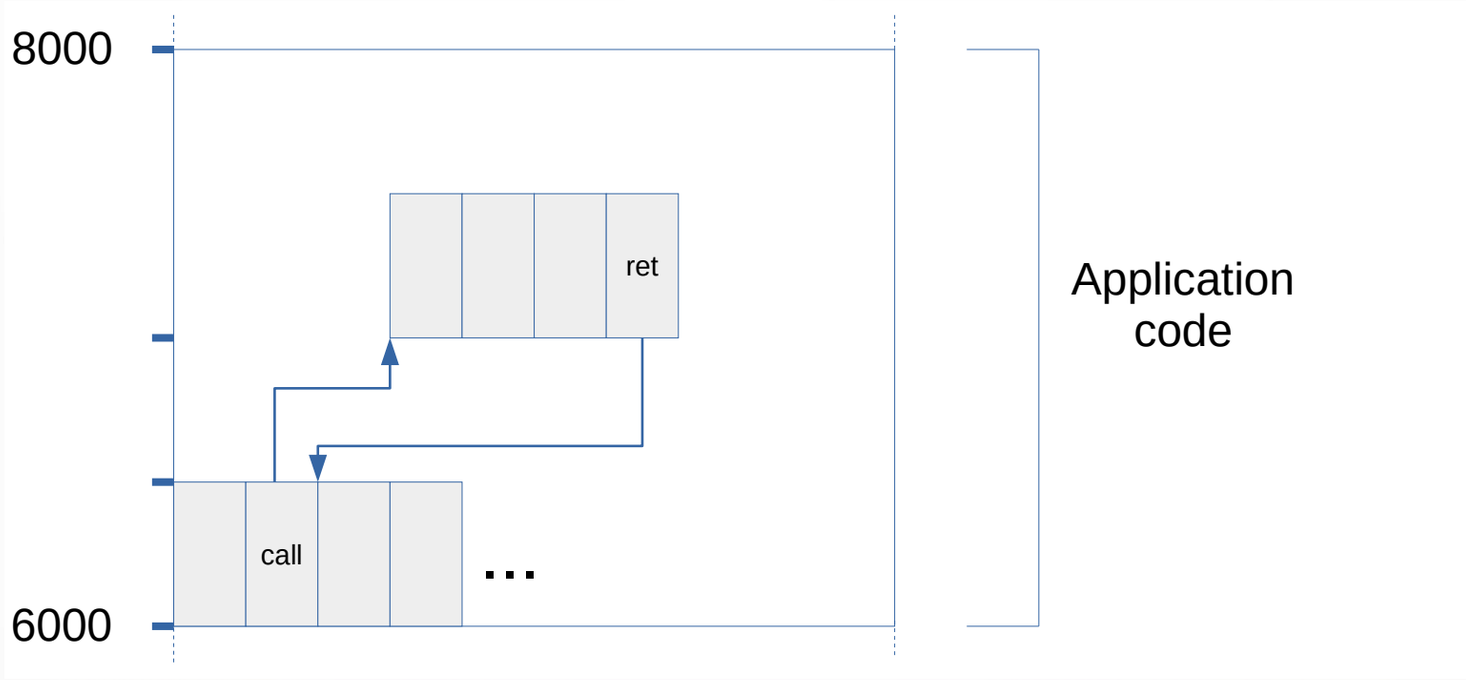
Writing to memory

- Space allocated to an application is divided into two parts:
  - MOV[8001], AL put the value contained in AL register (here 42) into memory at address 8001



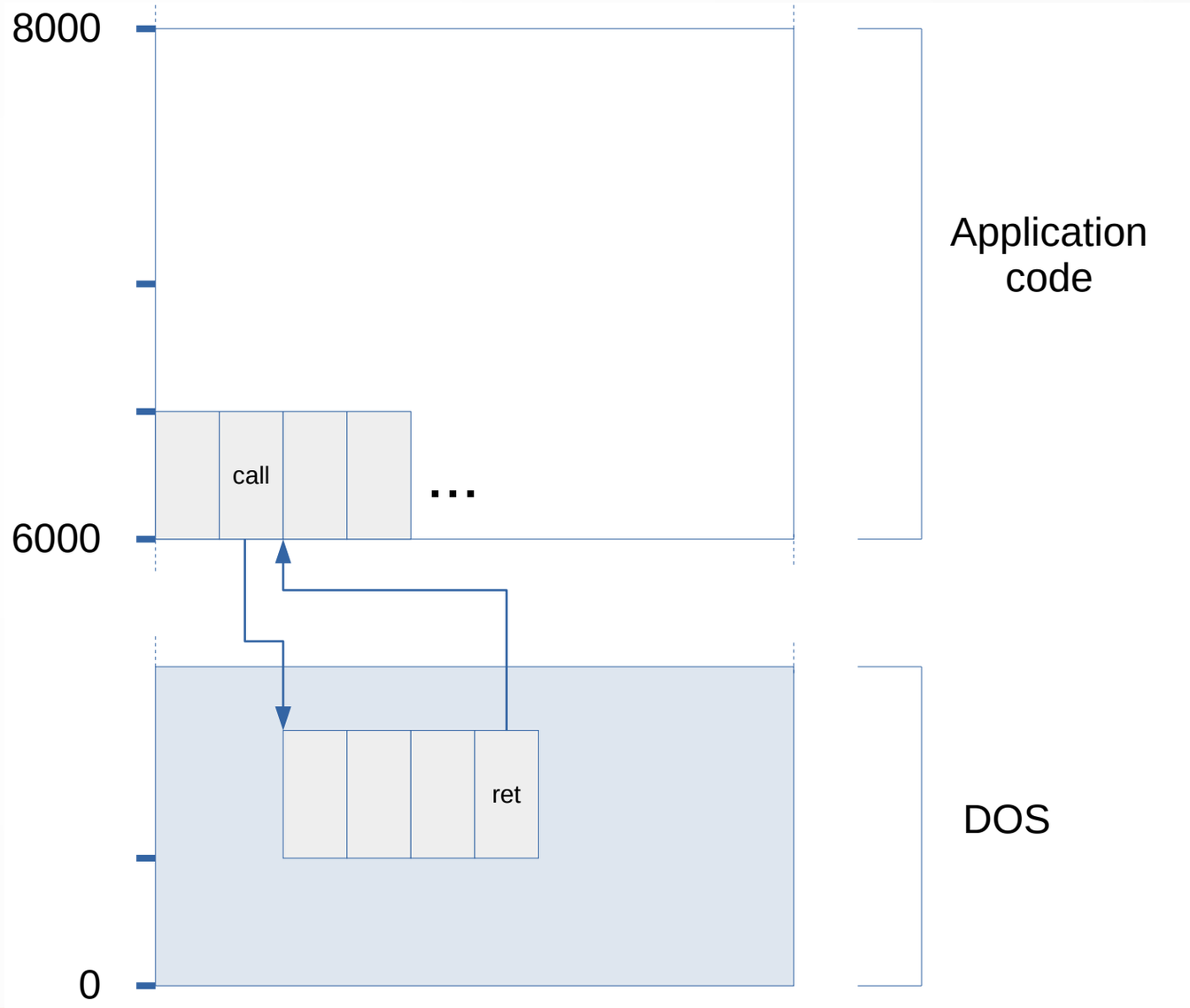
Calling a procedure

- A special **call** instruction can be used to jump to non-concomitant code.
  - After using this instruction, the execution continues at a new place in the application code.
  - When executing the **ret** instruction, the execution goes back the the instruction following the last **call**.



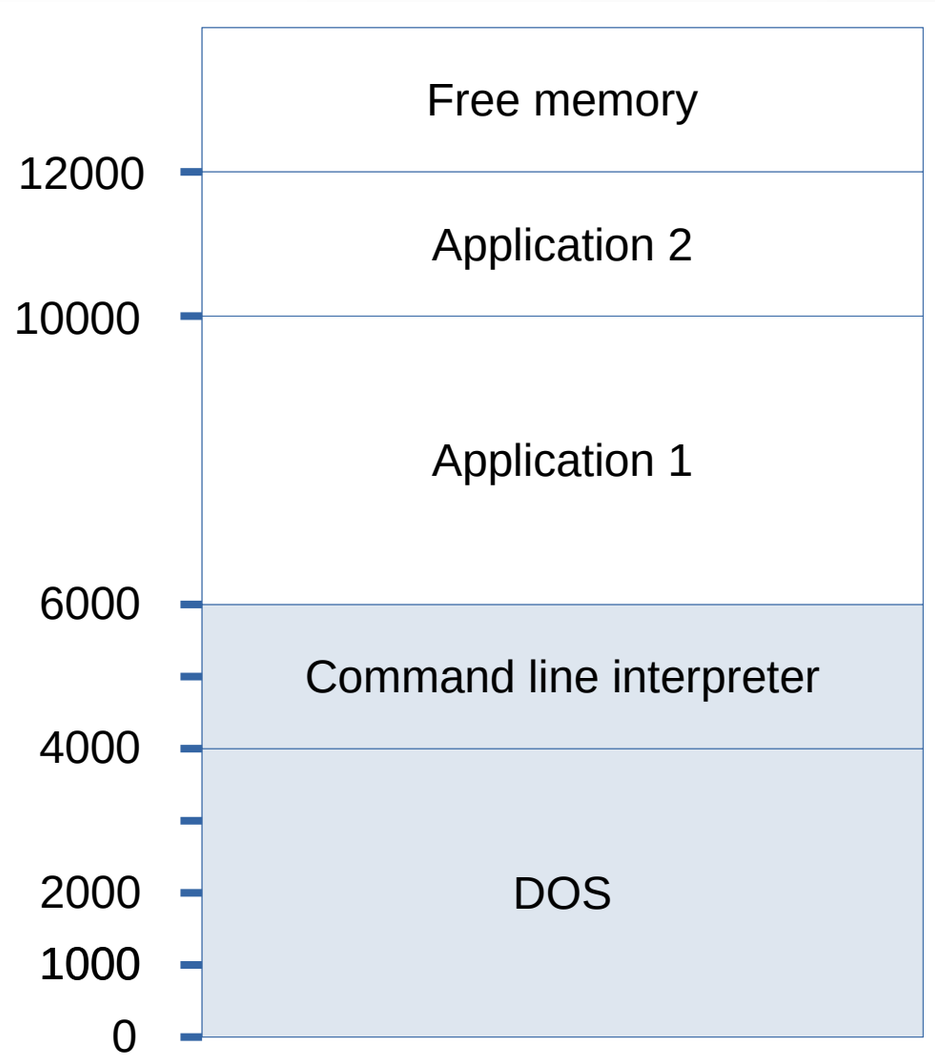
Making a system call

- System calls are basically calls made to procedures that reside into the code area of the system.
- You always need the procedure address to make a call.



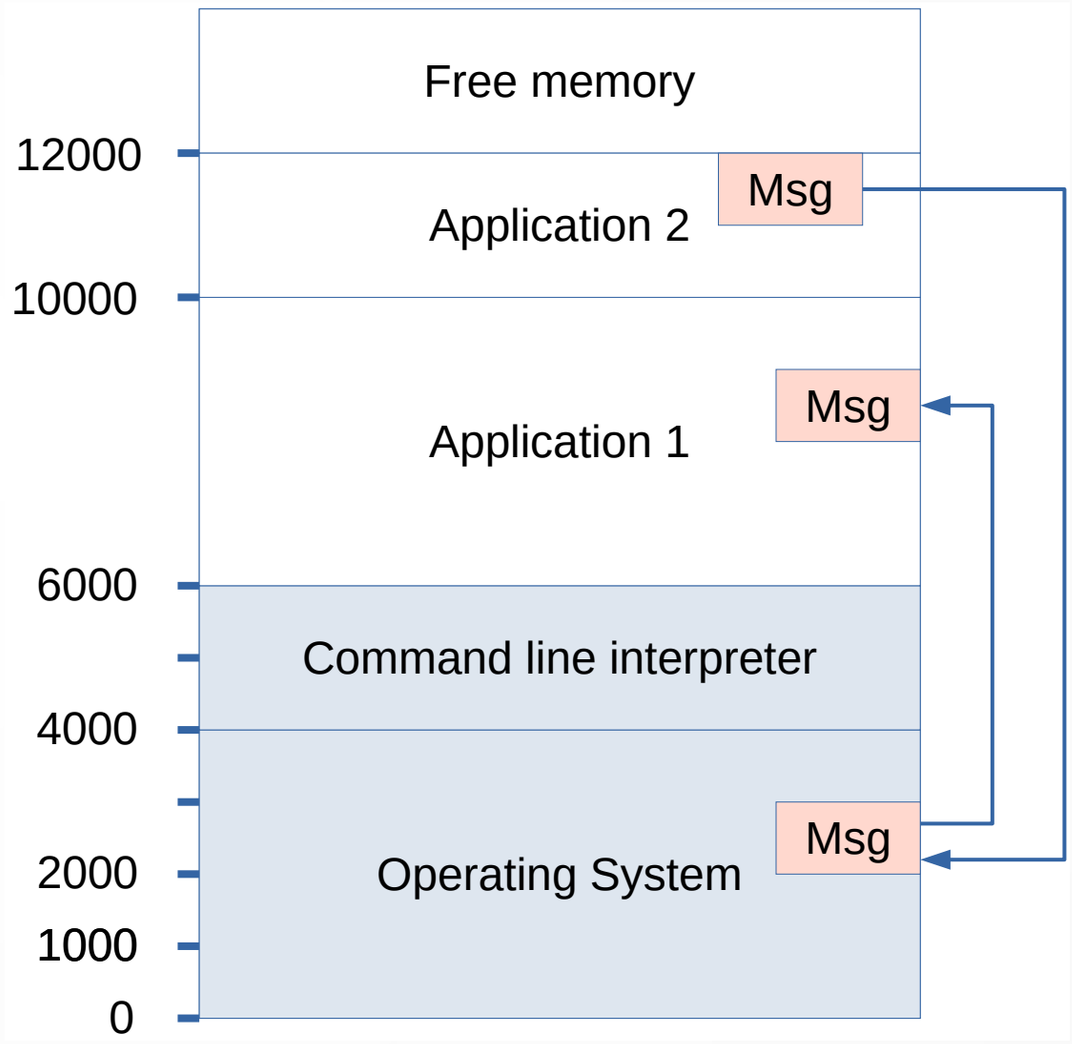
Multiple running applications

- Even in MS-DOS, it was possible to have multiple applications loaded at the same time.
- To operating system is responsible for choosing the address at which an application is loaded
- Everything is managed through the memory addresses.



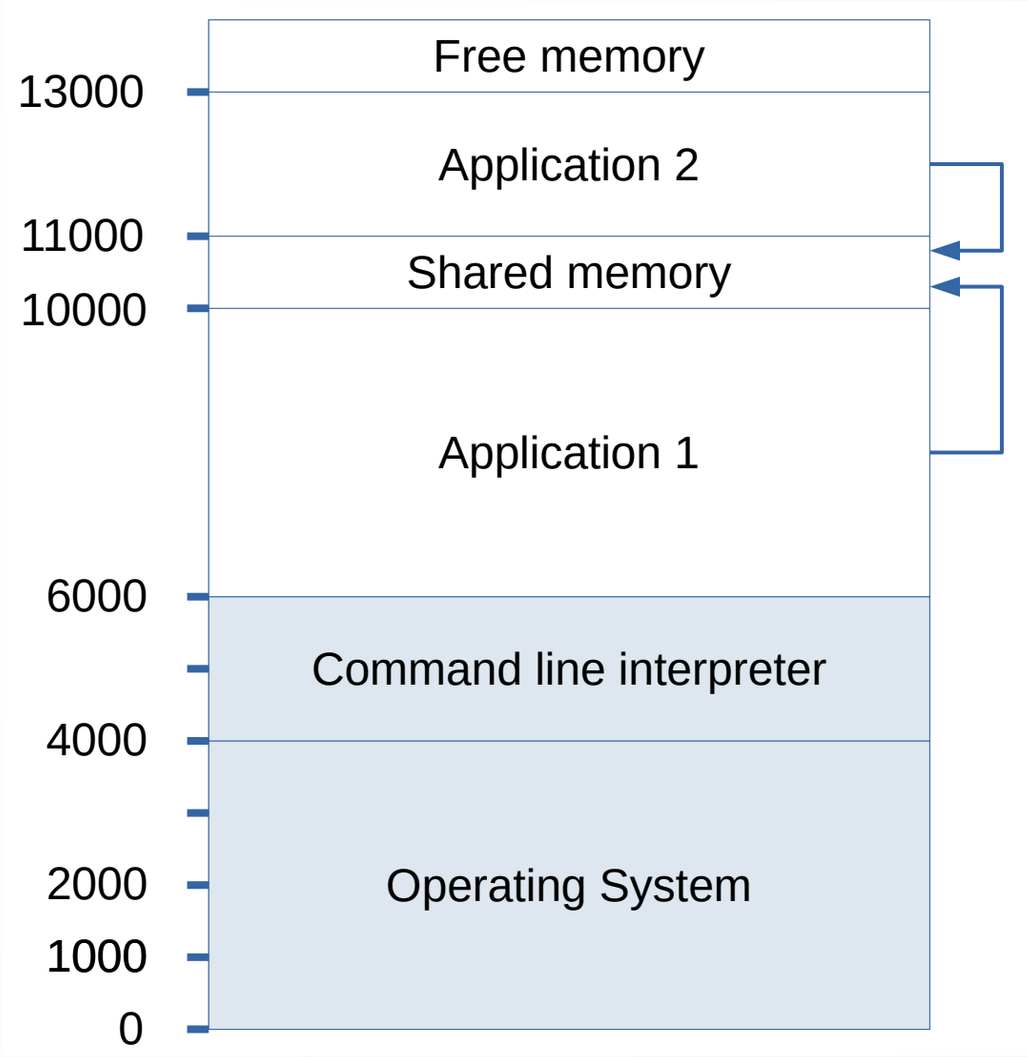
Communication models

- Communicating between applications is sometime required.
- This can be done by message passing
  - Application 2 create a message.
  - A call the the API of the operating system is made.
  - The operating system distribute the message to the receiving application.



Communication models

- Communicating between applications is sometime required.
- This can be done by shared memory
  - The two applications access to the same area
  - This requires mechanisms to prevent interferences (reading as the other is not finished writing for example)
  - Semaphores or mutex can by used to prevent problems.



Security implications

- A lot of problems arise in this type of memory management.
  - In case of programming error, an instruction can try to **read** from a incorrect address. Think for example when you mess with the index when traversing an array. The read instruction does not fail. It works, but it gets a wrong result !
  - In case of programming error, an instruction can try to **write** to an incorrect address. Instead of updating a variable, you may:
    - Update another variable
    - Change your own code (most probably causing your application to crash later)
    - Change data or code from another application, or even from the operating system !

Security implications

- A lot of problems arise in this type of memory management.
  - Using a wrong address may be done voluntarily
    - To access information from another application
      - Save the state of an application to restore it later (as in some old games with no save points)
      - Access private informations (a password stored in a variable)
    - To modify another application's data in memory
      - It was nice in some old games: restore health or set to max level, yeah!
    - To change other application's (or operating system) code in memory
      - The viruses do that a lot

Security implications

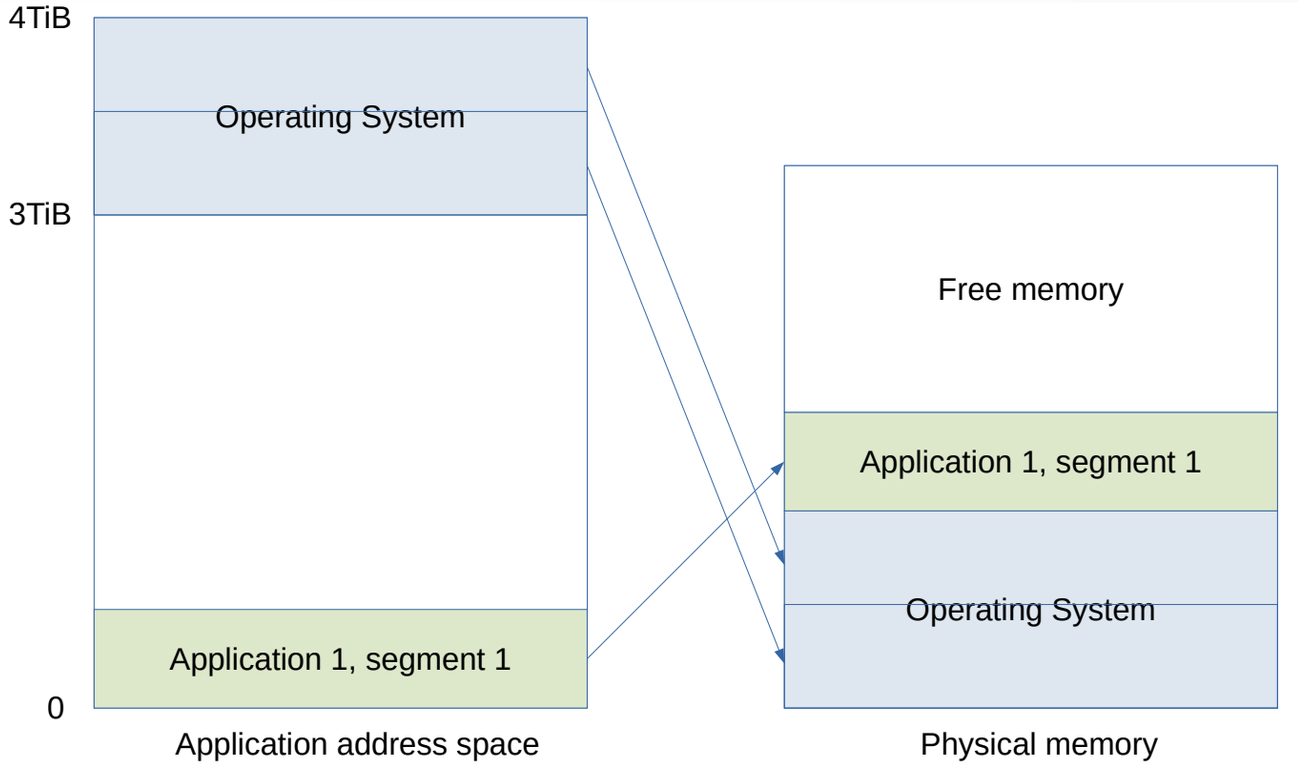
- In summary, this memory model is not safe.
- By mistake or voluntarily, it’s possible to access the whole memory from any application.
- Applications are not really isolated from others.
- Even the operating system can be (very easily) targeted.

Memory mapping and virtual memory

- To handle this problems, virtual memory has been used for many years.
- Each process has its own address space
  - Kernel is visible in this address space (in order to allow system calls)
  - Code and data from this process is also visible
  - All other processes are not visible at all
- Memory is fragmented into segments. One application may possess multiple segments.

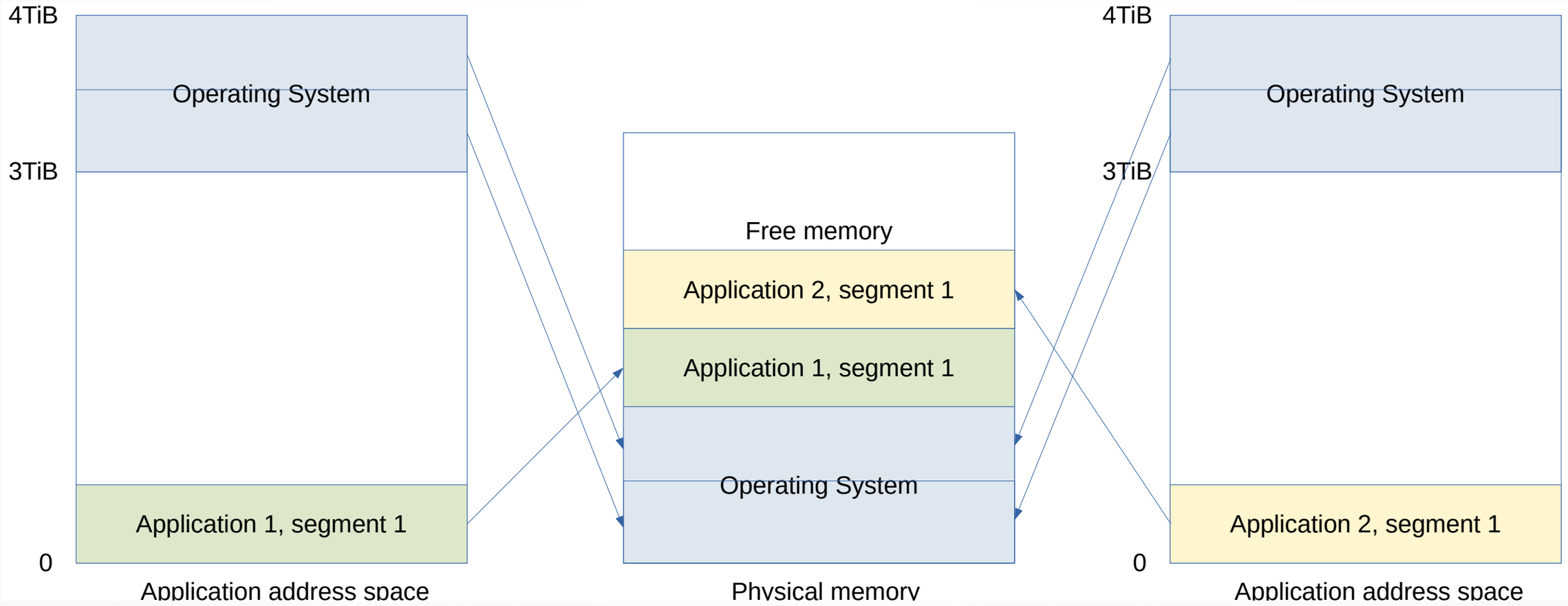
Memory mapping and virtual memory

- Segments (pages) from operating system are mapped to the upper region of user space.
- The application's page is mapped at the beginning of the user space.
- When the application want to access to data, it uses addresses from user space.
- **Memory accesses are intercepted** by the Memory Management Unit (MMU, a part of the CPU) and redirected to the correct page in the real memory space. This is done transparently to the application.



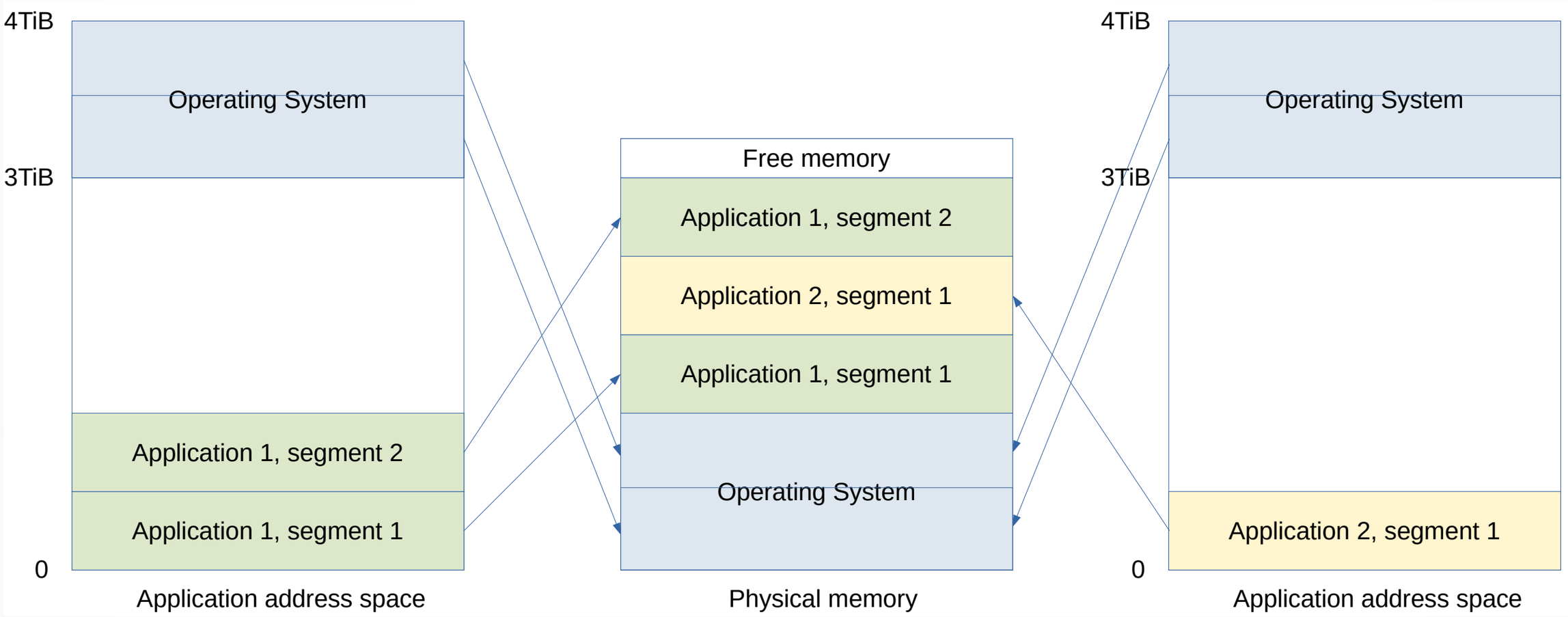
Memory mapping and virtual memory

- Another application will possess its own address space
- Its pages will be mapped somewhere else in the real memory
- In user space, operating system area is read-only (enforced by MMU)



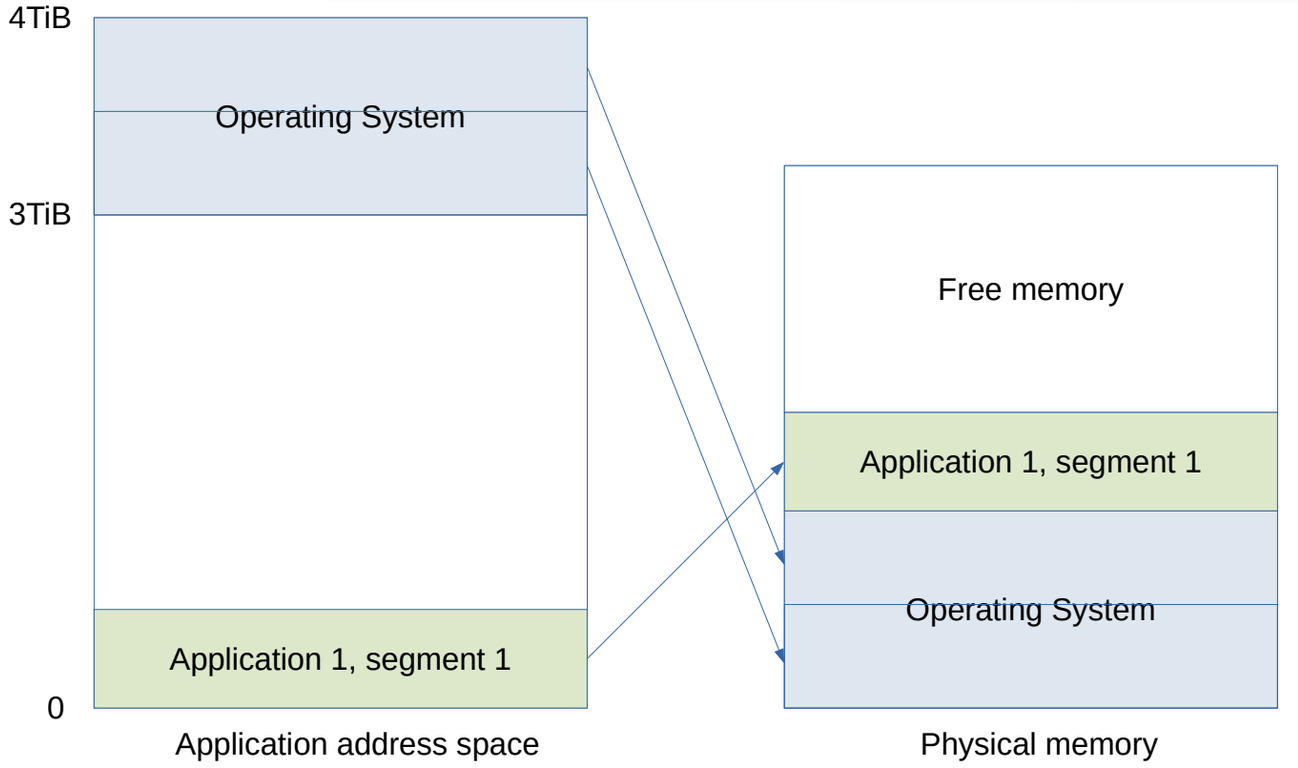
Memory mapping and virtual memory

- If application 1 needs more memory, it can ask the operating system (via a memory call, for example *malloc* on Linux)
- The operating system find some free space in the real memory and enable a user space memory mapping to it.
- If not enough real memory is available, the call will fail.



Memory mapping and virtual memory

- If your code does something illegal, the MMU will detect it.
  - You try to write in the operating system area
  - You try to read or write at an address that has no mapping to the physical memory
- In that case, the CPU will stop executing the faulty instruction in your code.
- It calls the operating system back, to let it decide what to do. Usually, it terminates your application. As you tried to access an unavailable segment, you get a *segmentation fault* error from the operating system.

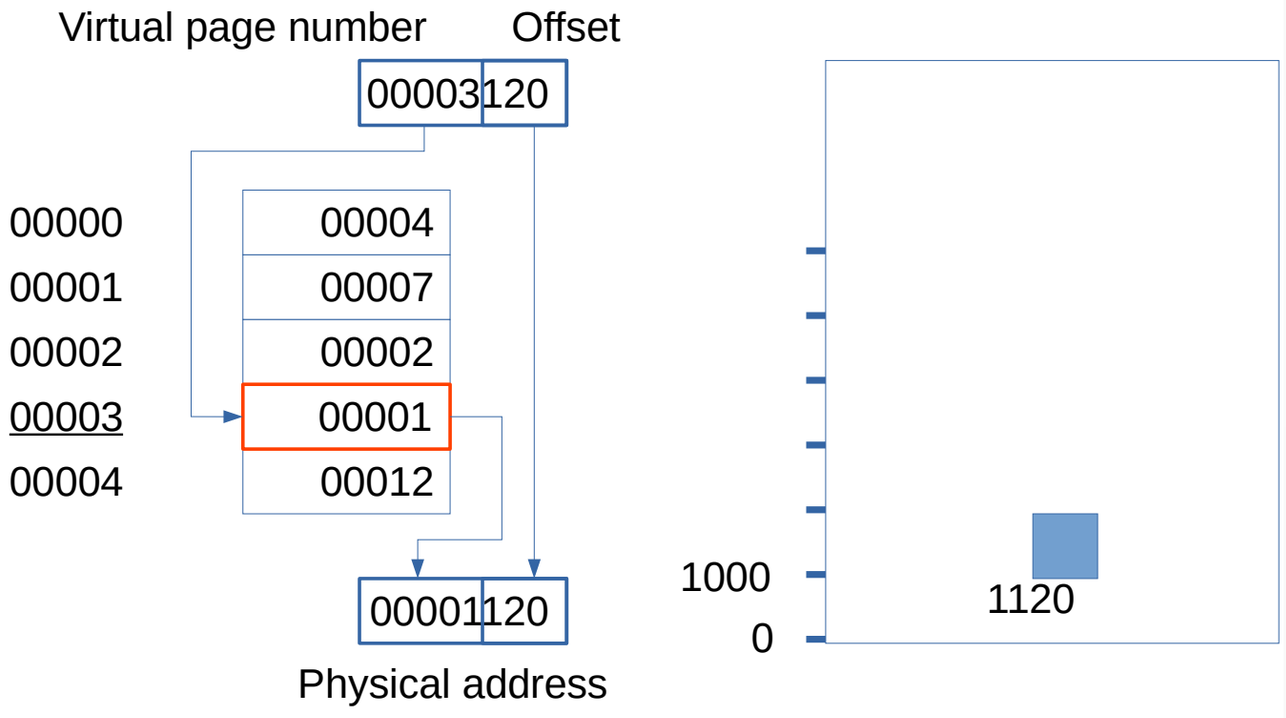


Memory mapping and virtual memory

- This system requires cooperation between:
  - The hardware MMU, which physically translate virtual (user) memory addresses into physical addresses.
  - The operation system, that manage the translation tables. The operating system is responsible for allocating the frames of physical memory to the processes. It build and maintain the tables that the MMU use to translate addresses.

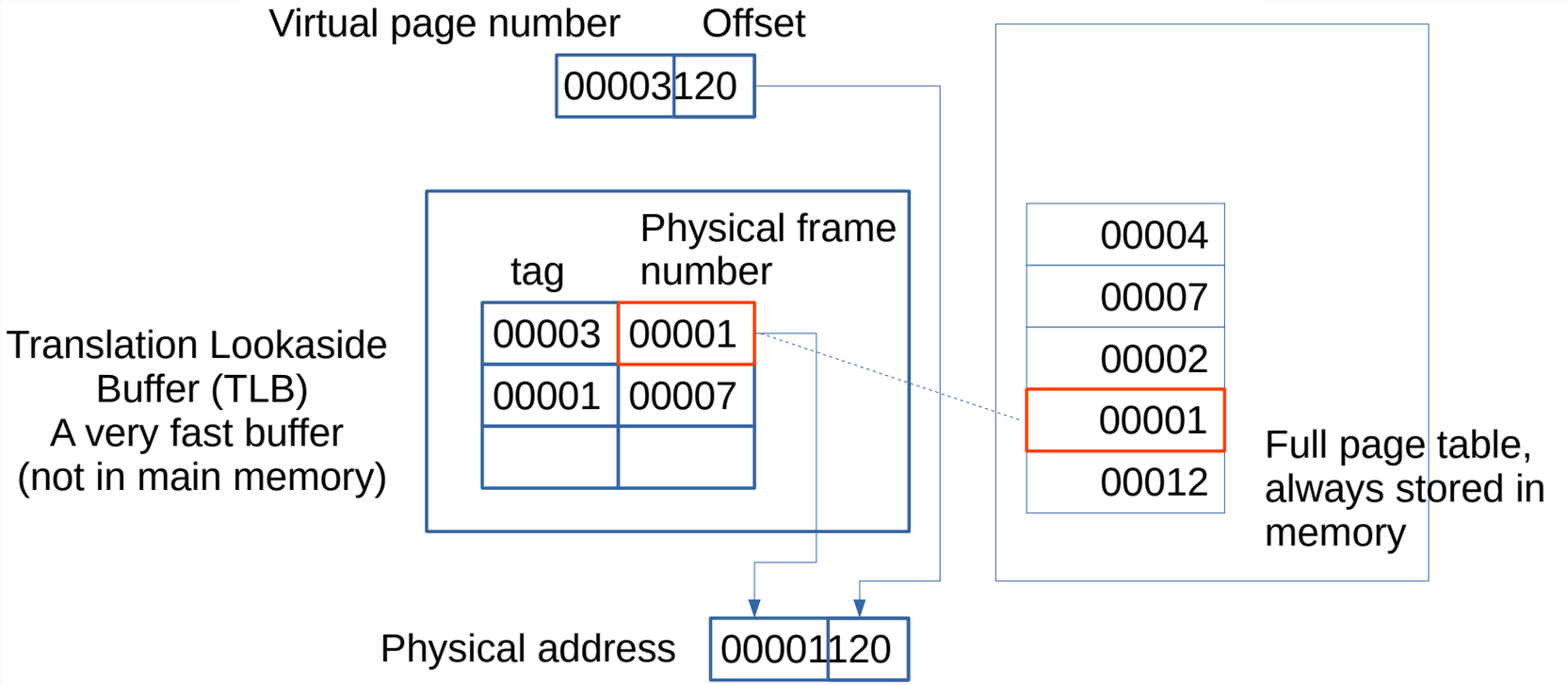
Memory mapping and virtual memory

- The mapping between the virtual and physical space is done through a page table.
- Each process has its own space so it must possess its own page table.
- A virtual address is divided into a virtual page number and an offset in the page.
- The virtual page number indicates the index to look at in the page table (here, page number 3, so we look at the 4th row).
- In the page table, we get the first part of the physical address (sometime called the frame number)
- We append the offset at the end and get the physical address.



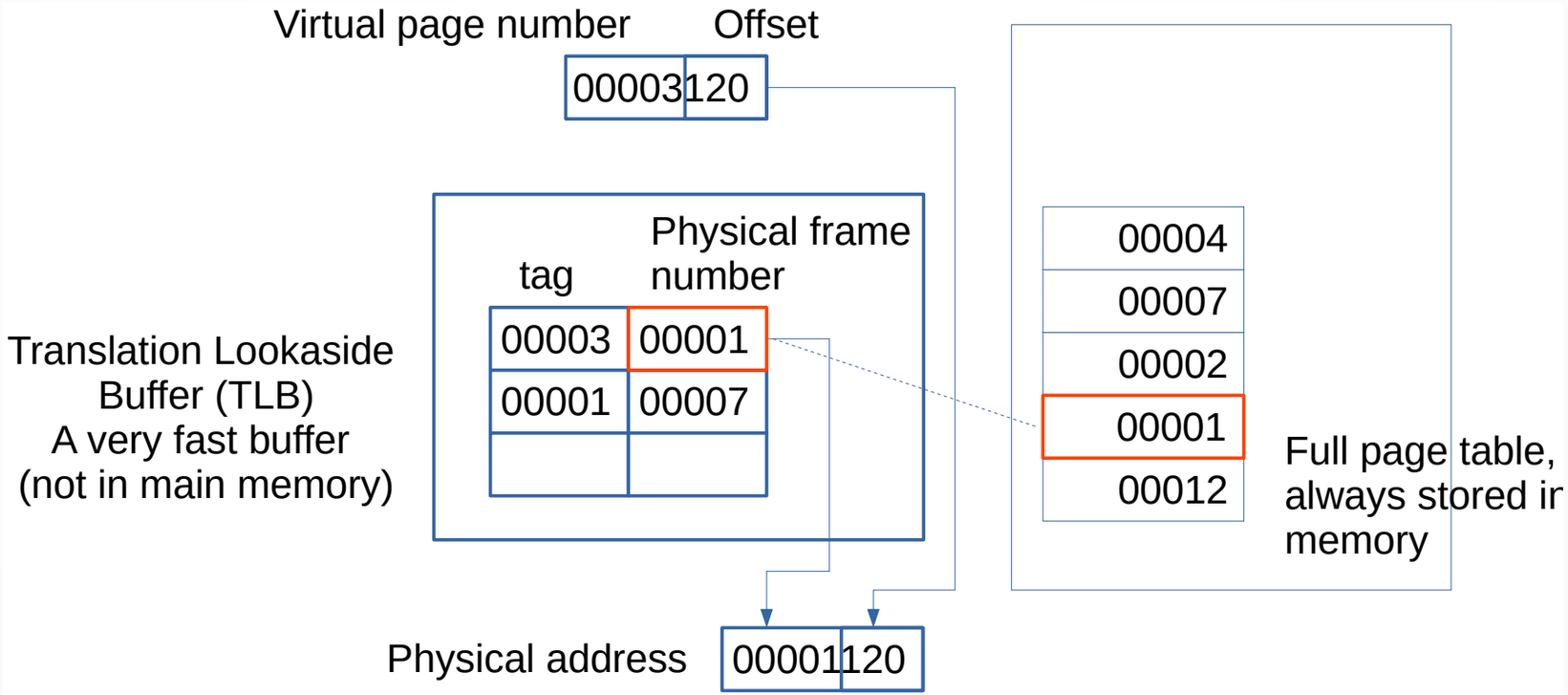
Memory mapping and virtual memory

- As the page table may be big, it can't be store completely in the fast general purpose cache of the microprocessor.
- Instead, microprocessors use a smaller but very fast cache called Translation Lookaside Buffer (TLB).
- It contains a limited number of entries



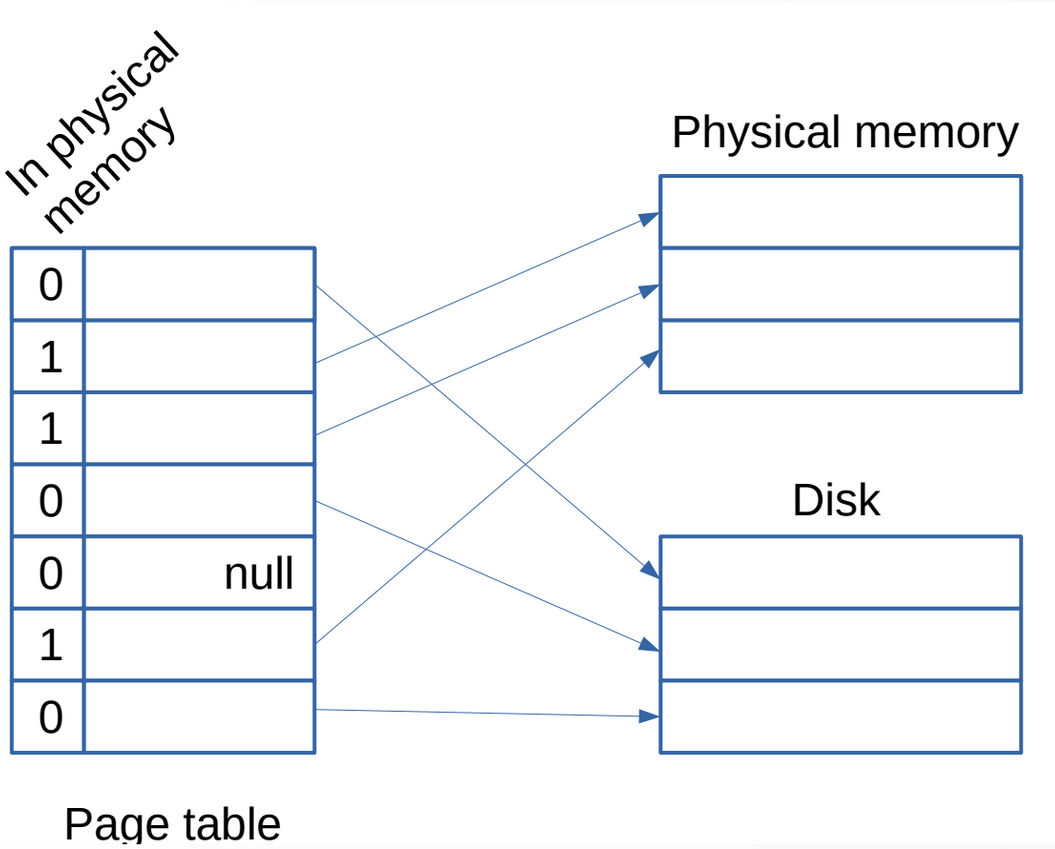
Memory mapping and virtual memory

- When the MMU needs to find the mapping to a physical address, it looks into the TLB.
- If the information is there, no need to search further.
- If it is not, it has to search in the full page table, store in the main memory. The main memory is slower, but when the correspondence is found, an entry is added to the TLB, to be faster next time.



Memory mapping and virtual memory

- The implementation of the page table is usually not as simple as previously depicted.
- An entry in the page table may be null (does not map to a physical address). This is the case when the virtual address space is bigger than the physical address space.
- A entry in the page table may map to a frame that is not currently in memory, but which is stored on a disk instead. This is often called **swap**, and allows to allocate more RAM than physically available.
- If we try to access to a page that is currently on disk, the operating system will take over and bring back to swapped frame to memory. The memory access will then resume transparently for the application (albeit at the cost of a significant delay).



Memory mapping and virtual memory

- How big is the page table ?
  - It depends on:
    - The size of the address space
    - The individual size of a page.
  - For a given address space (for example 4GiB), the smaller the size of a page, the higher the number of required entries in the page table.
- Having smaller pages prevent over-allocation and unused memory (pages are atomic, when an application needs 10 bytes more, it cannot get less than the size of a page).
- Having smaller pages requires a bigger page table, thus wasting memory.
- It's a trade-off. Different architectures may propose different values, an it may even be configured at operating system level. Smallest pages are usually 4KiB, but can be as big as 1GiB.

Memory mapping and virtual memory

Exemple:

- Me have a 32 bits address space (2^32 = 4GiB)
- Each entry in the page table is 4 bytes (32 bits)
- The size of a page is 4096 bytes (2^12)

– How big is the page table ?

- As the size of a page is 4096 bytes, this means we need 12 bits for the offset
- Given that a full address is 32 bits long, this means that the virtual page number is 32-12 = 20 bits long.
- This means we need 2^20 entries in the page table, as there are 2^20 possible values for the virtual page number.
- Given that the size of an entry is 4 bytes and we need 2^20 = ~1M, the page table needs to occupy 4MiB

Memory mapping and virtual memory

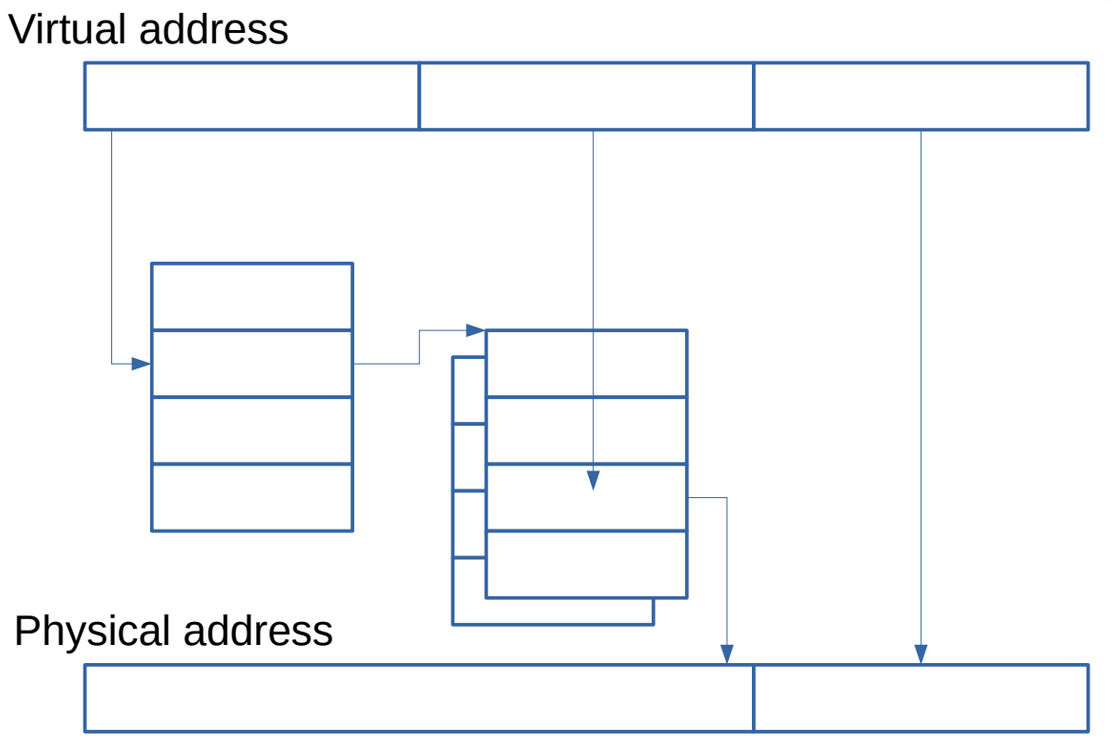
Application:

- We have a 64 bits address space
- Each entry in the page table is 8 bytes (64 bits)
- The size of a page is 4096 bytes (2^12)

– How big the page table for each process should be ?

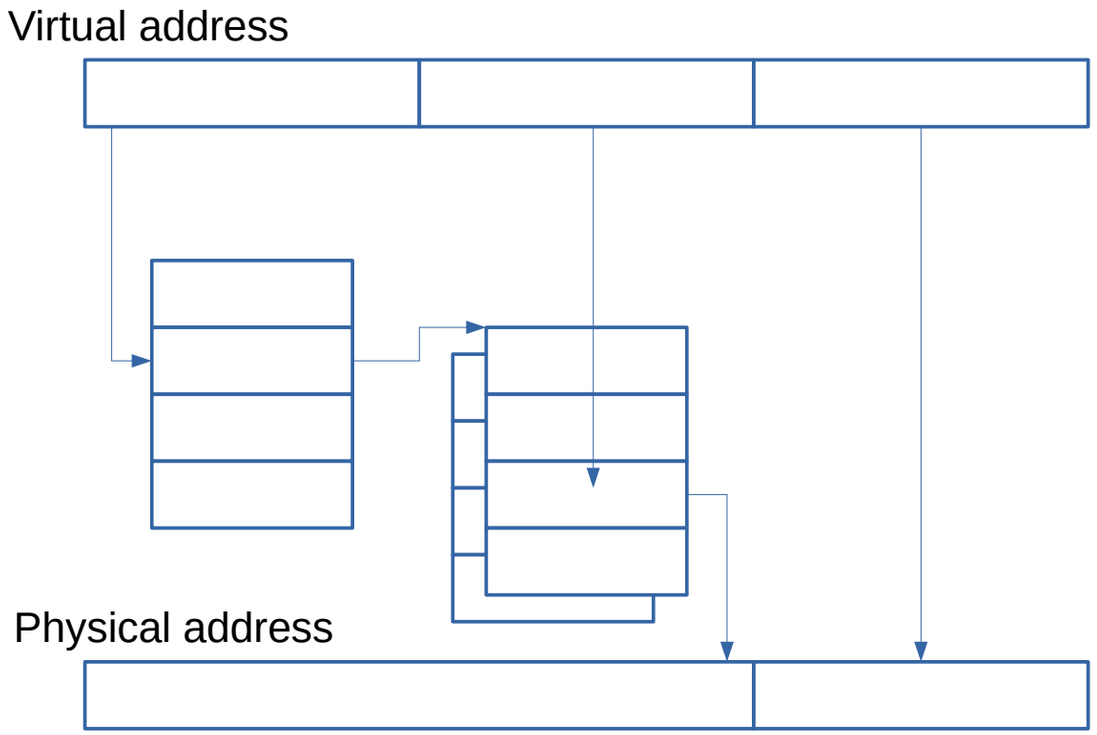
Memory mapping and virtual memory

- From the previous computation, it appears that large virtual address spaces cannot be handled by a single, very large page table. Please recall that a 64 bits virtual address space is much larger than the physical memory installed, so not all pages really need to be mapped.
- Instead of a single table, modern microprocessors use multi-levels page tables, where a virtual address is an association of 3 parts or more.
- The first part point to a table where entries can be null.



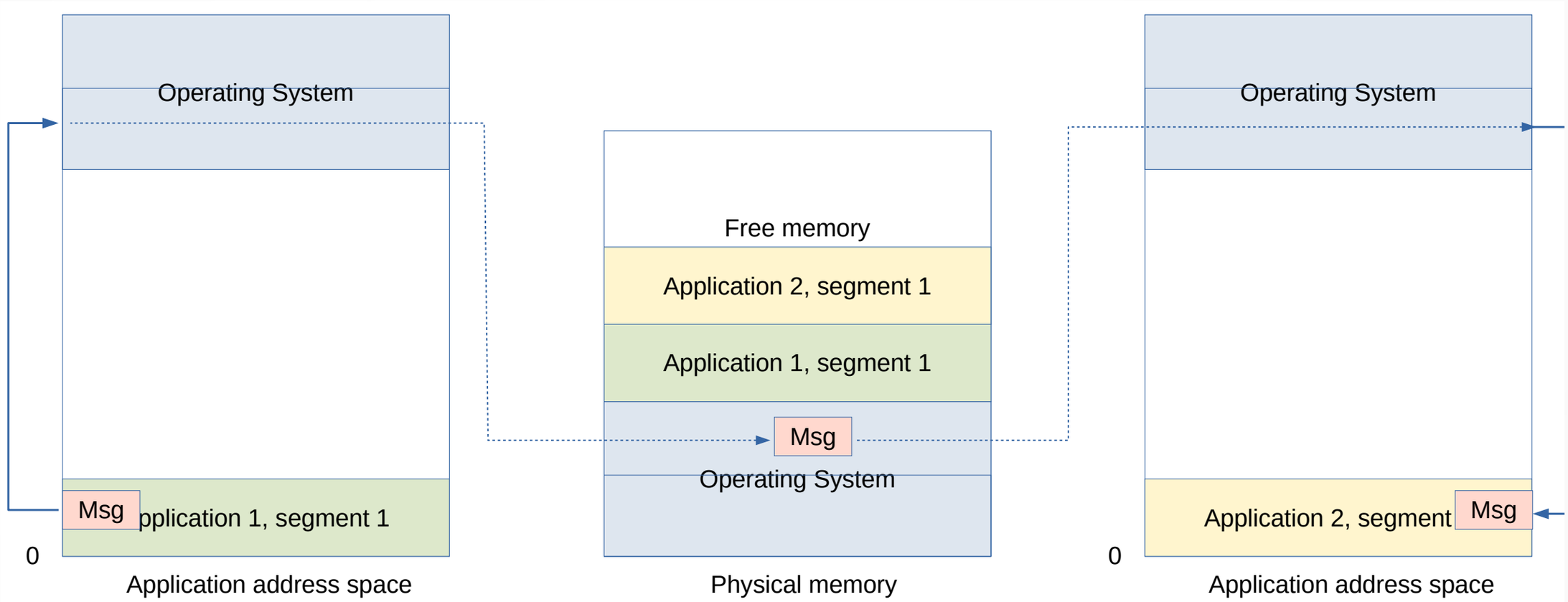
Memory mapping and virtual memory

- The first part point to a table where entries can be null. If not null, those entries points to others – second level – tables.
- In these second level table, the entry is chosen from the second part of the virtual address.
- In that way, we get the physical address without maintaining a full mapping of the huge virtual address space.
- In this system, the size of the page tables grows with the effective amount of memory effectively installed.
- As there are several level of indirection, this is done at a latency cost.



Memory mapping and virtual memory

- Message passing is allowed with this memory management
- The operating system provide the necessary API.



Memory mapping and virtual memory

- Shared memory is also working.
- Segments in the two processes map to the same physical frame.
- The operating system provide the necessary tools (mutex, semaphores, …).